

高级数据结构第七章习题答案

学号：202533354 姓名：ZHOU MINGYUAN

例题 1

题干：

有如下记录序列（按“成绩”作为关键字进行排序）：

[(A,85), (B,92), (C,85), (D,78), (E,92)]

回答：

- (1) 按成绩从小到大排序后，结果应是什么？
- (2) 在这个问题中，什么是“记录”，什么是“关键字”？
- (3) 如果排序后 A 仍在 C 前面，B 仍在 E 前面，这说明该排序算法具有什么性质？
- (4) 若数据量极大，无法全部装入内存，应属于内部排序还是外部排序？

答案：

- (1) 按成绩从小到大排序为 [(D,78), (A,85), (C,85), (B,92), (E,92)]。
- (2) 记录是 A、B、C、D、E 这些学生成绩条目；关键字是“成绩”。
- (3) 说明排序算法具有稳定性。
- (4) 数据量无法全部装入内存时属于外部排序。

解析：

稳定排序会保持相同关键字记录的原始相对次序。内部排序要求数据主要在内存中完成；外部排序用于数据规模超过内存容量的场景。

例题 2

题干：

对数组

[26, 5, 37, 1, 19]

应用插入排序。

回答：

- (1) 写出 $j=2,3,4,5$ 每一轮结束后的数组状态。
- (2) 哪一轮没有发生元素移动？
- (3) 最终排序结果是什么？

答案：

$j=2$ 后：[5, 26, 37, 1, 19]

$j=3$ 后：[5, 26, 37, 1, 19]

$j=4$ 后：[1, 5, 26, 37, 19]

$j=5$ 后：[1, 5, 19, 26, 37]

没有发生元素移动的是 $j=3$ 。

最终排序结果为 $[1, 5, 19, 26, 37]$ 。

解析：

插入排序每轮把当前元素插入前面已经有序的子数组中。37 已经大于前面元素，所以 $j=3$ 不需要移动；1 和 19 则需要向前插入。

例题 3

题干：

分别考虑下面两个数组：

$A = [1, 3, 5, 7, 9]$

$B = [9, 7, 5, 3, 1]$

回答：

- (1) 对 A 使用插入排序时，属于最好情况还是最坏情况？
- (2) 对 B 使用插入排序时，属于最好情况还是最坏情况？
- (3) 为什么插入排序对“近乎有序”的数据特别合适？
- (4) 插入排序的最好情况和最坏情况时间复杂度分别是什么？

答案：

- (1) A 属于最好情况。
- (2) B 属于最坏情况。
- (3) 近乎有序时，每个元素只需要很少移动，所以插入排序很合适。
- (4) 最好情况时间复杂度为 $O(n)$ ，最坏情况为 $O(n^2)$ 。

解析：

插入排序的代价主要来自向前比较和移动。数组已经有序时每轮只比较一次；逆序时每个元素都要移动到前面，移动次数达到平方级。

例题 4

题干：

对数组

$[23, 78, 45, 8, 32, 36]$

应用冒泡排序。

~~$[23, 45, 8, 32, 36, 78]$~~

~~$[23, 8, 32, 36, 45, 78]$~~

回答：

- (1) 写出第 1 趟结束后的数组状态。
- (2) 写出第 2 趟结束后的数组状态。

(3) 第 1 趟结束后, 哪个元素已经处在最终位置?

(4) 冒泡排序中, 已排序区间是从左边扩大还是从右边扩大?

答案:

(1) 第 1 趟结束后: [23, 45, 8, 32, 36, 78]。

(2) 第 2 趟结束后: [23, 8, 32, 36, 45, 78]。

(3) 第 1 趟结束后, 78 已经处在最终位置。

(4) 冒泡排序的已排序区间从右边向左扩大。

解析:

冒泡排序通过相邻比较与交换, 把当前未排序区间中的最大元素逐步交换到最右侧。因此每完成一趟, 右端就多一个最终确定的元素。

例题 5

题干:

一个长度为 n 的数组使用冒泡排序。

回答:

(1) 第 1 趟要比较多少次?

(2) 第 2 趟要比较多少次?

(3) 总比较次数可以写成什么求和式?

(4) 时间复杂度是什么?

答案:

(1) 第 1 趟比较 $n-1$ 次。

(2) 第 2 趟比较 $n-2$ 次。

(3) 总比较次数为 $(n-1)+(n-2)+\dots+1 = n(n-1)/2$ 。

(4) 时间复杂度为 $O(n^2)$ 。

解析:

每一趟都会把一个最大元素放到最终位置, 下一趟的未排序区间长度减少 1, 因此比较次数形成等差求和。

例题 6

题干:

对数组

[26, 5, 37, 1, 19, 24]

~~[1, 26, 5, 37, 19, 24]~~

~~[1, 5, 26, 37, 19, 24]~~

~~[1, 5, 19, 26, 37, 24]~~

应用选择排序。

回答：

- (1) 写出第 1 轮结束后的数组状态。
- (2) 写出第 2 轮结束后的数组状态。
- (3) 写出第 3 轮结束后的数组状态。
- (4) 选择排序每一轮“确定”的是什么？
- (5) 选择排序和冒泡排序相比，交换次数通常谁更多？

答案：

- (1) 第 1 轮结束后：[1, 26, 5, 37, 19, 24]。
- (2) 第 2 轮结束后：[1, 5, 26, 37, 19, 24]。
- (3) 第 3 轮结束后：[1, 5, 19, 26, 37, 24]。
- (4) 选择排序每一轮确定未排序区间中的最小元素及其最终位置。
- (5) 冒泡排序通常交换次数更多；选择排序比较多，但交换次数较少。

解析：

按题目给出的过程，每轮把当前最小值放到未排序区间最前端。选择排序的主要成本是寻找最小值时的比较，而不是频繁交换。

例题 7

题干：

考虑长度为 6 的数组使用选择排序。

回答：

- (1) 第 1 轮比较多少次？
- (2) 第 2 轮比较多少次？
- (3) 总比较次数是多少？
- (4) 选择排序的时间复杂度是什么？
- (5) 选择排序的主要特点是什么：比较多，还是交换多？

答案：

- (1) 第 1 轮比较 5 次。
- (2) 第 2 轮比较 4 次。
- (3) 总比较次数为 $5+4+3+2+1 = 15$ 。
- (4) 时间复杂度为 $O(n^2)$ 。
- (5) 选择排序的特点是比较次数多，但交换次数相对较少。

解析：

长度为 6 时，每轮都要在剩余未排序元素中寻找最小值，比较次数逐轮减少 1。

例题 8

题干：

对数组

[26, 5, 37, 1, 61, 11, 59, 15, 48, 19]

应用快速排序，规定每次都选当前子数组的第一个元素作为 pivot。

回答：

- (1) 第一次 pivot 是多少？
- (2) 第一次 partition 完成后，pivot 最终落在哪个位置？
- (3) 第一次 partition 后，pivot 左边的元素满足什么性质？
- (4) 第一次 partition 后，pivot 右边的元素满足什么性质？

答案：

- (1) 第一次 pivot 是 26。
- (2) 26 左边有 5 个元素比它小，因此 partition 完成后它在第 6 个位置（0-based 下标为 5）。
- (3) pivot 左边元素都小于 26。
- (4) pivot 右边元素都大于 26。

解析：

partition 的目标不是让左右子数组内部完全有序，而是把 pivot 放到最终排序位置，并保证左侧不大于 pivot、右侧不小于 pivot。

例题 9

题干：

仍对数组

[26, 5, 37, 1, 61, 11, 59, 15, 48, 19]

应用快速排序，第一次 partition 完成后，pivot=26 已归位。

回答：

- (1) 接下来要递归处理哪两个子数组？
- (2) 为什么 pivot 归位后不需要再参与后续排序？
- (3) 快速排序属于哪一种重要算法思想？

答案：

- (1) 接下来递归处理左子数组 [5, 1, 11, 15, 19] 和右子数组 [37, 61, 59, 48]（左右内部顺序可随 partition 实现不同而不同）。
- (2) pivot=26 已经处在最终位置，所以不再参与后续排序。
- (3) 快速排序属于分治法。

解析：

快速排序将问题分成“小于 pivot 的部分”和“大于 pivot 的部分”，再分别递归排序。pivot 一旦归位，它的最终排名已经确定。

例题 10

题干：

快速排序每次都选当前子数组的第一个元素作为 pivot。

回答：

- (1) 如果每次都能把数组大致平分，时间复杂度是什么？
- (2) 如果每次都分成 $n-1$ 和 0 两部分，时间复杂度是什么？
- (3) 哪种输入更容易使“取第一个元素作 pivot”的快速排序退化到最坏情况？
- (4) 为什么说 pivot 的选择很关键？

答案：

- (1) 若每次大致平分，时间复杂度为 $O(n \log n)$ 。
- (2) 若每次分成 $n-1$ 和 0，时间复杂度为 $O(n^2)$ 。
- (3) 已经有序或逆序的数据更容易使“取第一个元素作 pivot”的快速排序退化。
- (4) pivot 决定划分是否均衡，直接影响递归深度和总比较次数。

解析：

快速排序每层 partition 代价约为 $O(n)$ 。若递归深度是 $O(\log n)$ ，总复杂度为 $O(n \log n)$ ；若深度退化为 $O(n)$ ，总复杂度变为 $O(n^2)$ 。

例题 11

题干：

关于快速排序，回答下列问题：

- (1) 为什么递归实现的快速排序需要栈空间？
- (2) 在较理想情况下，递归深度大约是多少？
- (3) 在最坏情况下，递归深度可能达到多少？
- (4) 为什么“三数取中”比固定取第一个元素更合理？

答案：

- (1) 递归快速排序需要栈空间保存子问题边界、返回地址和局部变量。
- (2) 理想情况下递归深度约为 $O(\log n)$ 。
- (3) 最坏情况下递归深度可能达到 $O(n)$ 。
- (4) 三数取中比固定取第一个元素更容易选到接近中间的 pivot，减少极端划分概率。

解析：

pivot 选择越接近中位数，左右子数组越均衡。三数取中用局部采样改善 pivot 质量，尤其能缓解有序或近乎有序输入的退化。

例题 12

题干：

对数组

[59, 5, 11, 26, 77, 19, 1, 48, 61, 15]

应用归并排序。

回答：

- (1) 写出“分割”阶段的过程，直到每个子数组只剩一个元素。
- (2) 为什么子数组只剩一个元素时可以停止？
- (3) 归并排序和快速排序在“分”的阶段有什么明显不同？

答案：

(1) 分割过程：

[59, 5, 11, 26, 77, 19, 1, 48, 61, 15]

-> [59, 5, 11, 26, 77] 和 [19, 1, 48, 61, 15]

-> [59, 5]、[11, 26, 77]、[19, 1]、[48, 61, 15]

-> [59]、[5]、[11]、[26, 77]、[19]、[1]、[48]、[61, 15]

-> [59]、[5]、[11]、[26]、[77]、[19]、[1]、[48]、[61]、[15]

(2) 单个元素天然有序，可以停止分割。

(3) 归并排序按位置二分；快速排序按 pivot 分区，左右规模取决于 pivot。

解析：

归并排序的 divide 阶段不比较元素大小，只把数组持续拆半；真正的有序性在 merge 阶段建立。

例题 13

题干：

对下面两个有序数组进行合并：

左半部分：[5, 26, 59]

右半部分：[1, 15, 19, 48]

左半部分：[]

右半部分：[]

[1, 5, 15, 19, 26, 48, 59]

回答：

- (1) 逐步写出每次取出的元素。
- (2) 写出最终合并后的结果。
- (3) 为什么归并操作能在线性时间内完成？

答案：

- (1) 取出顺序为 1, 5, 15, 19, 26, 48, 59。
- (2) 合并结果为 [1, 5, 15, 19, 26, 48, 59]。
- (3) 归并操作能在线性时间完成，因为左右指针只向前移动，每个元素只被取出一次。

解析：

两个输入数组已经分别有序，因此每一步只需比较两个当前指针所指元素，把较小者放入输出序列。

例题 14

题干：

关于归并排序，回答：

- (1) 归并排序的基本思想是什么？
- (2) 它的时间复杂度是什么？
- (3) 它的最坏情况性能是否稳定？
- (4) 它相比堆排序，主要缺点是什么？

答案：

- (1) 基本思想是分而治之：先递归二分，再把有序子数组合并。
- (2) 时间复杂度为 $O(n \log n)$ 。
- (3) 最坏情况下仍保持 $O(n \log n)$ ，性能稳定。
- (4) 相比堆排序，主要缺点是需要 $O(n)$ 额外辅助空间。

解析：

归并排序每层合并总代价为 $O(n)$ ，层数为 $O(\log n)$ 。它的代价不依赖输入初始顺序，因此最坏情况也稳定。

例题 15

题干：

判断下面数组是否可以看作一个最大堆：

(1) [77, 61, 59, 48, 19, 11, 26, 15, 1, 5]

(2) [26, 61, 59, 48, 19, 11, 77, 15, 1, 5]

回答：

(1) 哪一个是最堆？

(2) 判断最大堆时要检查什么性质？

(3) 最大堆中最大的元素一定在哪个位置？

答案：

(1) 第一个数组是最堆，第二个数组不是最大堆。

(2) 判断最大堆要检查每个父节点都大于或等于其孩子节点。

(3) 最大堆中最大的元素一定在根节点，也就是数组下标 0 的位置。

解析：

数组表示的堆是完全二叉树。第二个数组的根 26 小于孩子 61 和 59，已经违反最大堆性质。

例题 16

题干：

对数组

[26, 5, 77, 1, 61, 11, 59, 15, 48, 19]

构建最大堆。

回答：

(1) 最后一个非叶子节点是谁？

(2) 建堆时为什么要从最后一个非叶子节点开始向前调整？

(3) 建成最大堆后的数组结果是什么？

答案：

(1) 最后一个非叶子节点是数组下标 4 (1-based 第 5 个位置) 的元素 61。

(2) 因为叶子节点天然满足堆性质，自底向上调整可以保证调整父节点时其子树已经是堆。

(3) 建成最大堆后的数组为 [77, 61, 59, 48, 19, 11, 26, 15, 1, 5]。

解析：

对长度 n 的 0-based 数组，最后一个非叶子节点为 $\text{floor}(n/2)-1$ 。建堆从该位置向前逐个向下调整。

例题 17

题干：

已知某数组已经构成最大堆：

[77, 61, 59, 48, 19, 11, 26, 15, 1, 5]

回答：

- (1) 第 1 轮交换哪两个元素？
- (2) 为什么交换后最后一个位置可以视为已排序？
- (3) 交换后为什么还要做 adjust？
- (4) 堆排序中，已排序区间从哪一边开始扩大？

答案：

- (1) 第 1 轮交换堆顶 77 和最后一个元素 5。
- (2) 77 是当前最大元素，交换到最后后已经处在升序排序的最终位置。
- (3) 交换后根节点可能破坏最大堆性质，需要对剩余堆做 adjust。
- (4) 堆排序中已排序区间从右边开始扩大。

解析：

堆排序反复把最大堆的根与当前堆尾交换，然后缩小堆范围并恢复堆性质。

例题 18

题干：

有记录序列：

[(A,3), (B,2), (C,3), (D,1)]

按关键字从小到大排序。

回答：

- (1) 如果排序后 A 仍在 C 前面，这说明算法具有什么性质？
- (2) 在本章所学算法中，插入排序是否稳定？
- (3) 归并排序是否稳定？
- (4) 快速排序通常是否稳定？
- (5) 堆排序通常是否稳定？

答案：

- (1) 如果排序后 A 仍在 C 前面，说明算法稳定。
- (2) 插入排序是稳定的。
- (3) 归并排序在相等元素优先取左半部分是稳定的。
- (4) 快速排序通常不稳定。
- (5) 堆排序通常不稳定。

解析：

稳定性只讨论关键字相等的记录是否保持原始相对次序。会进行远距离交换或不区分相等元素来源的算法通常不稳定。

例题 19

题干：

现在有以下几种场景：

甲：数据量很小，而且基本已经有序；

乙：希望平均情况下尽量快；

丙：希望最坏情况下也保持较好性能；

丁：希望借助堆这种结构来排序。

请在插入排序、快速排序、归并排序、堆排序中，为每个场景选出最合适的一种，并说明理由。

答案：

甲：选择插入排序，因为数据量小且基本有序时移动少。

乙：选择快速排序，因为平均情况下通常很快，复杂度为 $O(n \log n)$ 。

丙：选择归并排序，因为最坏情况下也保持 $O(n \log n)$ 。

丁：选择堆排序，因为它正是利用堆结构完成排序。

解析：

算法选择应匹配输入规模、初始有序程度、最坏情况要求和空间限制。没有一种排序算法在所有约束下都最优。

例题 20

题干：

回答下列问题：

(1) 为什么说“基于比较”的排序可以用决策树来描述？

(2) n 个不同元素一共有多少种最终排列结果？

(3) 这说明像归并排序、堆排序、快速排序的 $O(n \log n)$ 性能处于什么水平？

答案：

(1) 基于比较的排序可用决策树描述，因为每次比较都会产生“是/否”分支。

(2) n 个不同元素共有 $n!$ 种最终排列。

(3) $O(n \log n)$ 的比较排序已经达到比较排序下界的同阶水平，是渐近最优的。

解析：

决策树必须至少有 $n!$ 个叶子来区分所有可能排列，因此高度至少为 $\log_2(n!)$ ，也就是 $\Omega(n \log n)$ 。